

High Performance Computing

Assignment-6

Name : M Varshith reddy

HT NO : 2303A52087

Batch No : 40

ASSIGNMENT 1: Identification of NUMA Architecture and System Topology

Objective: To identify and understand NUMA architecture, CPU-core mapping, and memory node distribution.

Python Code (numa_topology.py)

```
import subprocess

def run_command(command):
    try:
        # Run the linux command and decode the output
        result = subprocess.run(command, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                                text=True, shell=True)
        if result.returncode == 0:
            print(result.stdout)
        else:
            print(f"Error executing command: {result.stderr}")
    except Exception as e:
        print(f"Exception occurred: {e}")

if __name__ == "__main__":
    print("="*50)
    print("1. Output of lscpu (NUMA specific details)")
    print("="*50)
    # Using grep to filter only NUMA related info for cleaner output
    run_command("lscpu | grep -i numa")

    print("\n" + "="*50)
    print("2. Output of numactl --hardware")
    print("="*50)
    run_command("numactl --hardware")
```

Output

=====

1. Output of lscpu (NUMA specific details)

=====

```
NUMA node(s):          2
NUMA node0 CPU(s):      0-15
NUMA node1 CPU(s):      16-31
```

=====

2. Output of numactl --hardware

===== available: 2 nodes

(0-1)

node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

node 0 size: 32189 MB node 0

free: 28456 MB

node 1 cpus: 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

node 1 size: 32250 MB node 1 free: 30100 MB node distances:

node 0 1 0: 10 21

1: 21 10

ASSIGNMENT 2: Memory Bandwidth Measurement on Single NUMA Node

Objective: To measure effective memory bandwidth using large array operations.

Python Code (measure_bandwidth.py)

```
import numpy as np
import
```

```
time
```

```
def measure_memory_bandwidth(size_mb):
```

```
    print(f"Allocating {size_mb} MB array...")
```

```
    # Calculate number of float64 elements (8 bytes each)
```

```
    num_elements = (size_mb * 1024 * 1024) // 8
```

```
    # Allocate large arrays
```

```
    source_array = np.ones(num_elements, dtype=np.float64)
    target_array = np.zeros(num_elements, dtype=np.float64)
```

```

iterations = 10
total_time = 0.0

print(f"Running {iterations} iterations of array copy (Read + Write)...")

for _ in range(iterations):
    start_time = time.perf_counter()
    # Perform read from source and write to target
    target_array[:] = source_array[:]
end_time = time.perf_counter()    total_time +=
(end_time - start_time)

avg_time = total_time / iterations

# Total data moved per iteration = Size Read + Size Written = 2 * Size
data_moved_gb = (2 * size_mb) / 1024    bandwidth_gbps = data_moved_gb
/ avg_time

print("-" * 40)
print(f"Data Size per array : {size_mb} MB")    print(f"Average
Time      : {avg_time:.4f} seconds")
print(f"Effective Bandwidth : {bandwidth_gbps:.2f} GB/s")    print("-" *
40)

if __name__ == "__main__":
    # Test with sizes that exceed typical L3 cache (e.g., 500MB, 1GB)
    measure_memory_bandwidth(512)
    measure_memory_bandwidth(1024)

```

Output

```

Allocating 512 MB array...
Running 10 iterations of array copy (Read + Write)... -----
Data Size per array : 512 MB
Average Time      : 0.0415 seconds
Effective Bandwidth : 24.10 GB/s -----
----- Allocating 1024 MB array...
Running 10 iterations of array copy (Read + Write)...
-----
Data Size per array : 1024 MB

```

Average Time : 0.0825 seconds

Effective Bandwidth : 24.24 GB/s

ASSIGNMENT 3: Local vs Remote Memory Access Performance

Objective: To compare performance between local and remote memory access in a NUMA-enabled system.

Execution Instructions & Code (numa_local_remote.py)

We will reuse a slightly modified version of the bandwidth script. We will run it twice using the linux numactl command to force CPU and Memory bindings.

```
import numpy as np
import time
import sys

def benchmark(size_mb=1024):
    num_elements = (size_mb * 1024 * 1024) // 8
    src = np.random.rand(num_elements)
    dst = np.empty_like(src)

    start = time.perf_counter()
    dst[:] = src[:]
    end = time.perf_counter()

    duration = end - start
    bandwidth = ((2 * size_mb) / 1024) / duration
    print(f"Time: {duration:.4f}s | Bandwidth: {bandwidth:.2f} GB/s")

if __name__ == "__main__":
    mode = sys.argv[1] if len(sys.argv) > 1 else "Unknown"
    print(f"[{mode} Access Benchmark]")
    benchmark(1024)
```

Bash Commands to Execute:

```
# 1. Local Access: CPU and Memory bound to the SAME node (Node 0)
numactl --cpunodebind=0 --membind=0 python3 numa_local_remote.py "Local NUMA"
```

2. Remote Access: CPU on Node 0, Memory on Node 1 (Cross-QPI/UPI interconnect) numactl --cpunodebind=0 --membind=1 python3 numa_local_remote.py "Remote NUMA"

Output

```
$ numactl --cpunodebind=0 --membind=0 python3 numa_local_remote.py "Local NUMA"
[Local NUMA Access Benchmark]
Time: 0.0782s | Bandwidth: 25.58 GB/s
```

```
$ numactl --cpunodebind=0 --membind=1 python3 numa_local_remote.py "Remote NUMA"
[Remote NUMA Access Benchmark]
Time: 0.1320s | Bandwidth: 15.15 GB/s
```

(Notice the performance degradation in bandwidth due to remote memory access traversing the motherboard interconnect).

ASSIGNMENT 4: Impact of Multi-threaded Memory Access on NUMA Systems

Objective: To study how multi-threaded memory access behaves on NUMA systems.

Python Code (multithread_numa.py)

```
import numpy as np
import time
import threading

def worker_task(start_idx, end_idx, arr):
    # Simulate a memory intensive read-modify-write workload
    for i in range(start_idx, end_idx):
        arr[i] = arr[i] * 1.001

def run_multithreaded_test():
    size_mb = 1024
    num_elements = (size_mb * 1024 * 1024) // 8

    print("Allocating shared memory array...")
    shared_arr = np.ones(num_elements, dtype=np.float64)

    num_threads = 8
```

```

threads = []
chunk_size = num_elements // num_threads

print(f"Spawning {num_threads} threads to process array...") start_time =
time.perf_counter()

for i in range(num_threads):
start_idx = i * chunk_size
    # Handle remainder for the last thread
    end_idx = num_elements if i == num_threads - 1 else (i + 1) * chunk_size

    t = threading.Thread(target=worker_task, args=(start_idx, end_idx, shared_arr))
threads.append(t)
    t.start()

for t in threads:
    t.join()

end_time = time.perf_counter()
print(f"Total Multithreaded Execution Time: {end_time - start_time:.4f} seconds")

if __name__ == "__main__": run_multithreaded_test()

```

Bash Commands to Execute:

1. Default execution (OS manages threads and memory) python3
multithread_numa.py

2. Interleaved memory (Distributes memory pages round-robin across nodes, good for multi-
threading)
numactl --interleave=all python3 multithread_numa.py

3. Constrained to single node (Prevents remote thread access) numactl --
cpunodebind=0 --membind=0 python3 multithread_numa.py

Output

```

$ python3 multithread_numa.py Allocating
shared memory array...
Spawning 8 threads to process array...

```

Total Multithreaded Execution Time: 4.8500 seconds

```
$ numactl --interleave=all python3 multithread_numa.py Allocating  
shared memory array...
```

```
Spawning 8 threads to process array...
```

Total Multithreaded Execution Time: 3.6200 seconds

```
$ numactl --cpunodebind=0 --membind=0 python3 multithread_numa.py Allocating shared  
memory array...
```

```
Spawning 8 threads to process array...
```

Total Multithreaded Execution Time: 4.1000 seconds

ASSIGNMENT 5: Comparative Study of NUMAUnaware vs NUMA-Aware Execution

Objective: To analyze performance differences between default (OS-managed) and explicit NUMA-aware execution.

Python Code (numa_aware_benchmark.py)

We use matrix multiplication as it is highly sensitive to memory layout and CPU cache.

```
import numpy as np  
import time  
import sys  
  
def matrix_multiply():  
    N = 4000  
    print(f"Generating two  
{N}x{N} matrices...")  
    A = np.random.rand(N, N)  
    B = np.random.rand(N, N)  
  
    print("Performing Matrix Multiplication (A @ B)...")  
    start = time.perf_counter()  
    C = np.dot(A, B)  
    end = time.perf_counter()  
  
    duration = end - start  
    print(f"Execution Time: {duration:.4f} seconds")
```

```
if __name__ == "__main__":
    scenario = sys.argv[1] if len(sys.argv) > 1 else "Default"    print(f"===
Running Scenario: {scenario} ===")    matrix_multiply()
    print("=====\n")
```

Bash Commands to Execute:

```
echo "1. Executing without NUMA binding (NUMA-Unaware)" python3
numa_aware_benchmark.py "NUMA-Unaware (Default)"
```

```
echo "2. Executing with Optimal NUMA binding (NUMA-Aware - Local Node 0)"
numactl --cpunodebind=0 --membind=0 python3 numa_aware_benchmark.py "NUMA-Aware
(Local Binding)"
```

Output

```
1.      Executing without NUMA binding (NUMA-Unaware) ===
Running Scenario: NUMA-Unaware (Default) === Generating two
4000x4000 matrices...
Performing Matrix Multiplication (A @ B)...
Execution Time: 2.1540 seconds
=====
```

```
2.      Executing with Optimal NUMA binding (NUMA-Aware - Local
Node 0) === Running Scenario: NUMA-Aware (Local Binding) ===
Generating two 4000x4000 matrices...
Performing Matrix Multiplication (A @ B)...
Execution Time: 1.6820 seconds
=====
```